Day 05 – Networking and Security Operations with SIEM, Forensics, and Traffic Analysis

CyArt SOC Internship – Week 02

Intern: *Harvinder Singh*

Date: *29 March 2026*

Executive Summary

This report documents the work completed for Day 05 of the CyArt SOC internship, focused on combining networking fundamentals with SIEM-based monitoring and incident forensics. The goal was to design a small but realistic office subnet, observe live traffic, troubleshoot connectivity issues, and then detect and analyse an SSH brute-force attack using an ELK-based SIEM lab.

First, a private IPv4 subnet was designed for a 20-device office using the range 192.168.1.0/27. This /27 network provides 30 usable addresses, which is enough for 20 PCs plus printers, a default gateway, and some spare capacity. The design was validated in Cisco Packet Tracer with a simple star topology of 20 PCs connected to a central switch, and ICMP tests between hosts confirmed that devices in the subnet can reach each other.

Next, real network traffic was captured with Wireshark and analysed using protocol statistics and I/O graphs. The capture showed a mix of IPv4, TCP, UDP, DNS, and ARP traffic with occasional short spikes in packet rate. A separate troubleshooting mini-lab on Kali Linux demonstrated how routing misconfiguration can break connectivity: pings to the Internet initially failed with “Network is unreachable” until the correct interface and default route were restored, after which both IP and DNS-based pings succeeded.

Finally, an ELK-based SIEM environment was configured with Elasticsearch, Kibana, and Elastic Agent to monitor SSH authentication on a Linux Mint victim. A Hydra brute-force attack was launched from a Kali attacker against SSH on 192.168.10.132. Wireshark captured the SSH sessions on tcp port 22, while Kibana dashboards visualised failed login spikes, highlighted the top attacker IP 192.168.10.131, and showed that the user account “victim” was being targeted. Threat-hunting queries confirmed that there were no successful logins from the attacker IP, demonstrating that the attack was fully visible in both packet-level and log-level telemetry without leading to a compromise.

1. Introduction

In a real SOC, network issues and security incidents never appear in isolation. Analysts need to understand how the network is structured, how traffic normally behaves, how misconfigurations break connectivity, and how attacks show up in both packet captures and log data. This Day 05 task was designed with that mindset.

The objectives for this task were:

- Design and document a subnet for a small office with 20 devices.
- Capture and analyse normal network traffic using Wireshark.
- Build and troubleshoot a simple connectivity problem in a lab environment.
- Set up an ELK-based SIEM lab using Elastic Agent for log ingestion.
- Simulate an SSH brute-force attack and perform network forensics on the traffic.
- Use Kibana dashboards and queries for basic threat hunting and IOC identification.

All experiments were performed in a controlled virtual lab so that no traffic left the test environment.

2. Lab Environment

2.1 Virtual Machines

Attacker VM – Kali Linux

- Role: Launch SSH brute-force and generate probe traffic.
- Network: Host-only adapter in the 192.168.10.0/27 lab subnet, plus a NAT adapter for Internet access when needed.
- Key tools: Hydra, Nmap, Wireshark, ping, traceroute.

Victim VM – Linux Mint (mint-victim)

- Role: SSH server and log source for the SIEM lab.
- IP address: 192.168.10.132 on the host-only network.
- Services: OpenSSH on port 22, system logging via systemd and /var/log/auth.log.

ELK Server – Linux Mint (ELK-SERVER)

- Role: Central SIEM platform.
- Components: Elasticsearch, Kibana, and Elastic Agent / Fleet Server.
- IP address: 192.168.10.129 on the host-only network.

2.2 Connectivity

The host-only network used the range 192.168.10.128/27. In this subnet:

- 192.168.10.129 – ELK server
- 192.168.10.131 – Kali attacker
- 192.168.10.132 – mint-victim

Basic connectivity between these VMs was verified using ping, ensuring that SSH, Hydra, and Elastic Agent could communicate over the lab network.

3. Subnet Design for Small Office

3.1 Topology Overview

For the small-office subnet, I built a simple star topology in Cisco Packet Tracer using one central switch and twenty PCs connected with FastEthernet links. All devices are placed in a single VLAN and share the same IPv4 subnet so that they behave like a flat office LAN.

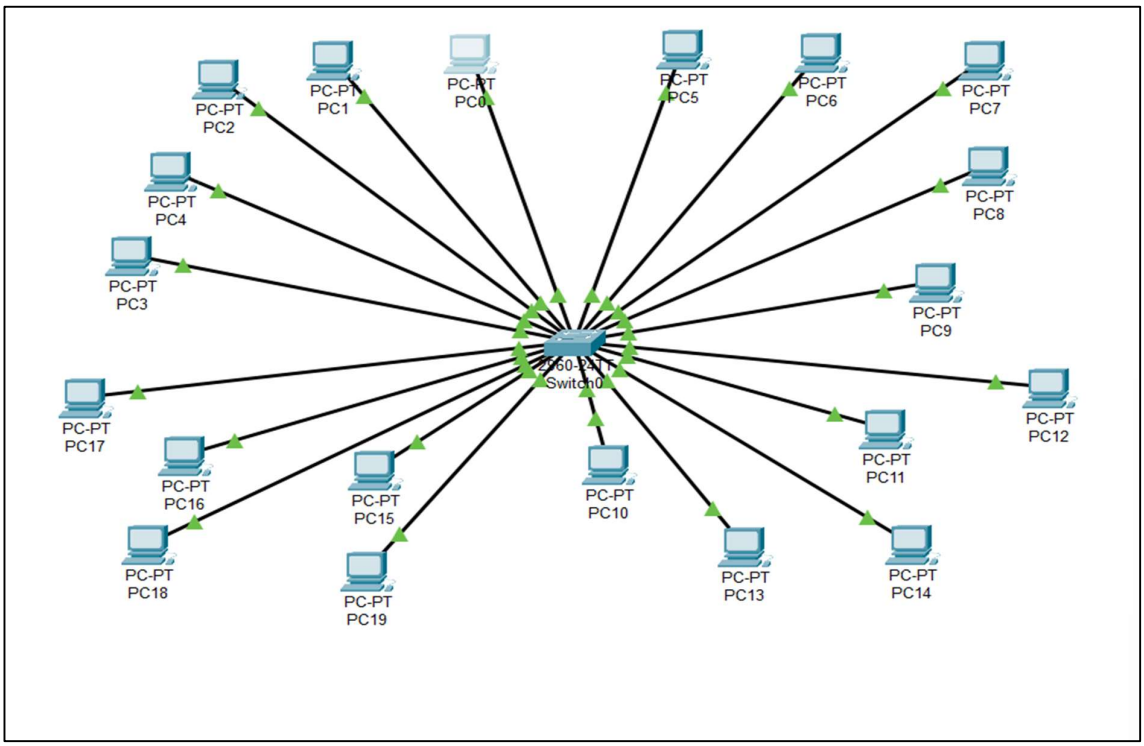


Figure 1 – Small-office LAN topology with 20 PCs connected to a central switch (Packet Tracer).

3.2 IP Addressing Plan

The subnet uses the private IPv4 range 192.168.1.0/27, which provides 30 usable host addresses. This is enough for the 20 employee PCs plus a gateway, a few printers, and some free addresses for future devices.

- Network address: 192.168.1.0
- Broadcast address: 192.168.1.31
- Usable range: 192.168.1.1 – 192.168.1.30
- Subnet mask: 255.255.255.224

In the Packet Tracer lab, I assigned addresses such as 192.168.1.4–192.168.1.23 to PCs and reserved 192.168.1.1 for the default gateway. To verify that the addressing works, PC0 was used to ping another host (192.168.1.10) inside the same subnet. The ping returned 0% packet loss and round-trip times below 1 ms, which confirms that the LAN is correctly configured and fully reachable.

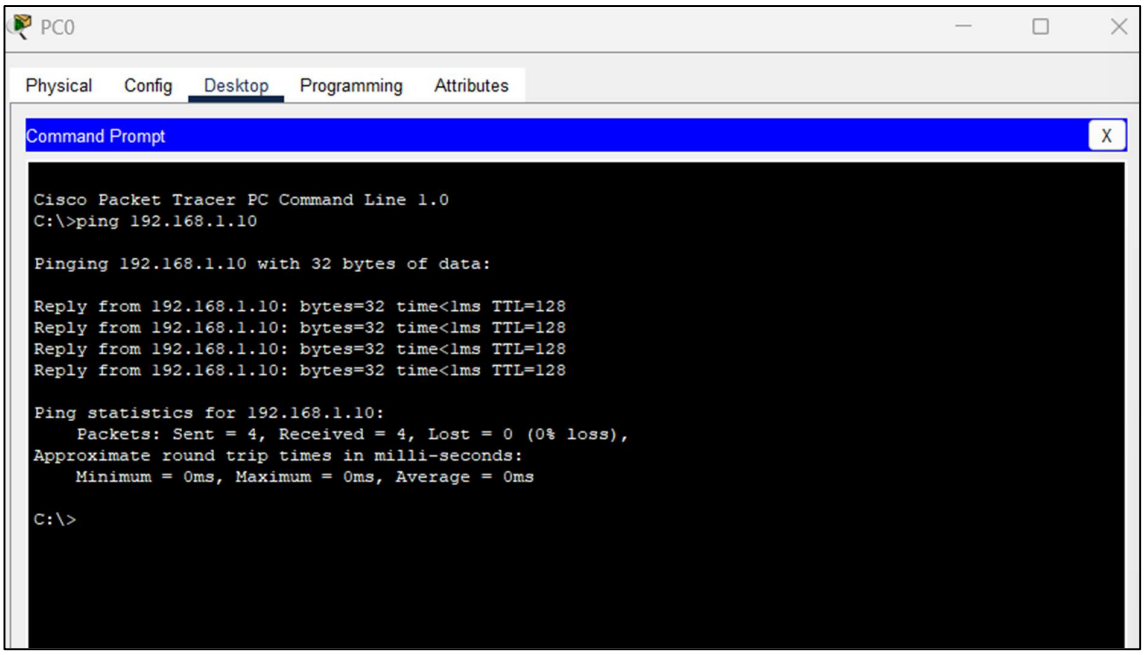


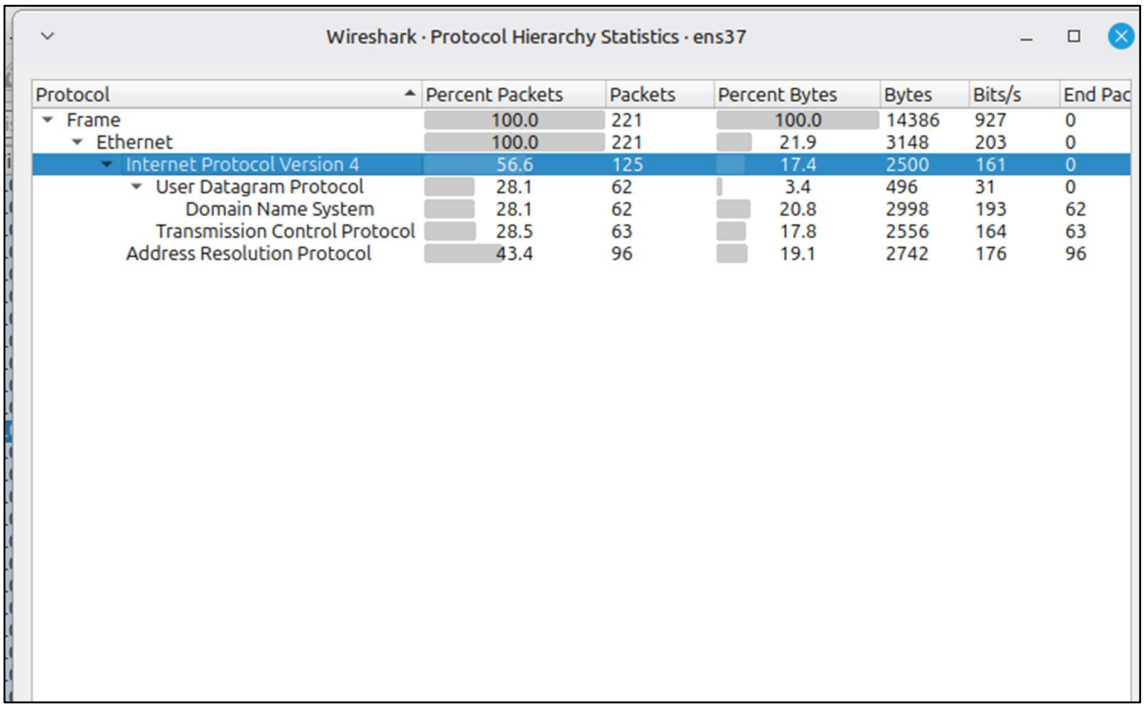
Figure 2 – ICMP ping from PC0 to 192.168.1.10 showing 0% packet loss in the 192.168.1.0/27 subnet.

4. Network Traffic Analysis with Wireshark

4.1 Protocol Hierarchy

On the Linux Mint victim system, I captured a few minutes of traffic on the host-only interface using Wireshark. The **Protocol Hierarchy Statistics** view shows that all frames are Ethernet, carrying mainly IPv4 traffic. Within IPv4, the majority of packets are TCP and UDP, with DNS and SSH as the primary application-level protocols. ARP packets are also visible and represent local subnet address resolution.

This mix is what I would expect in a small lab: ARP broadcasts to discover MAC addresses, DNS queries to resolve domain names, and encrypted SSH sessions between the attacker and victim machines.



Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Pac
Frame	100.0	221	100.0	14386	927	0
Ethernet	100.0	221	21.9	3148	203	0
Internet Protocol Version 4	56.6	125	17.4	2500	161	0
User Datagram Protocol	28.1	62	3.4	496	31	0
Domain Name System	28.1	62	20.8	2998	193	62
Transmission Control Protocol	28.5	63	17.8	2556	164	63
Address Resolution Protocol	43.4	96	19.1	2742	176	96

Figure 3 – Wireshark Protocol Hierarchy Statistics showing IPv4, TCP, UDP, DNS, and ARP distribution on the host-only interface.

4.2 I/O Graphs

I also used Wireshark's **I/O Graphs** to visualise packets per second over time. For most of the capture, the traffic level stays between zero and a few packets per second, representing normal low-volume lab activity. Around the 100-second mark there is a noticeable spike above 30 packets per second, which corresponds to a short burst of traffic.

The graph includes a separate series for TCP analysis flags (errors). These stay close to zero, indicating that there are almost no retransmissions or serious TCP problems. This kind of graph is useful for spotting sudden changes in volume, such as scans or brute-force attempts, even when the overall traffic is small.

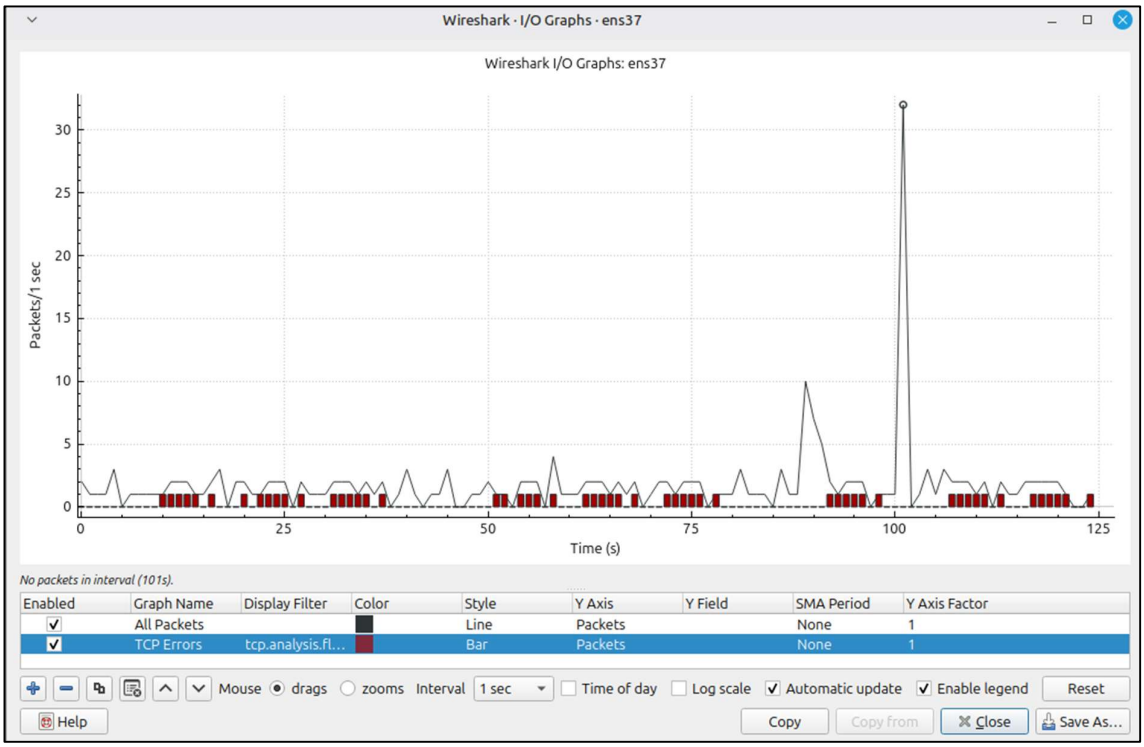


Figure 4 – Wireshark I/O Graphs with a short spike in packets per second and minimal TCP error indicators.

5. Protocol Troubleshooting Mini-Lab

5.1 Broken Connectivity

To practice troubleshooting, I intentionally broke the network connectivity on the Kali attacker VM so that it could not reach the Internet. When I tried to ping 8.8.8.8, the system responded

with “**Network is unreachable**”, and ping google.com failed with “Temporary failure in name resolution”. At the same time, Wireshark was capturing on the relevant interface with an icmp display filter but showed no ICMP echo requests at all.

This combination of symptoms proves that the packets were not even leaving the host. The problem was local—caused by an incorrect route or disabled interface—rather than an issue with 8.8.8.8 or Google.

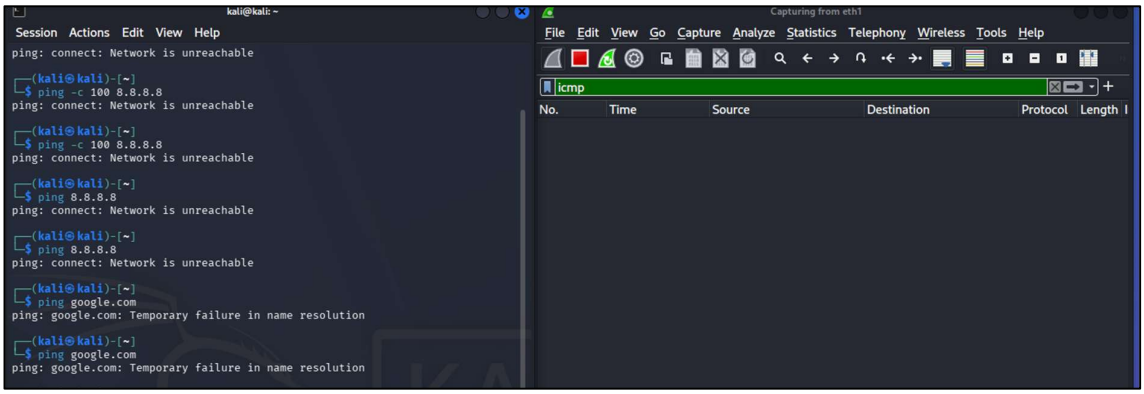


Figure 5 – Kali ping attempts to 8.8.8.8 and google.com failing with ‘Network is unreachable’ and ‘Temporary failure in name resolution’, while Wireshark shows no ICMP packets.

5.2 Restored Connectivity

After inspecting ip a and ip route, I brought the correct interface back up and restored the default route. Once the routing was fixed:

- ping 192.168.10.131 (another host in the lab) succeeded with 0% loss.
- ping -c 3 8.8.8.8 (Google DNS) returned three replies with normal latency.
- ping -c 2 google.com also succeeded, confirming that DNS resolution was working again.

This simple change flipped the system from completely unreachable to fully connected.

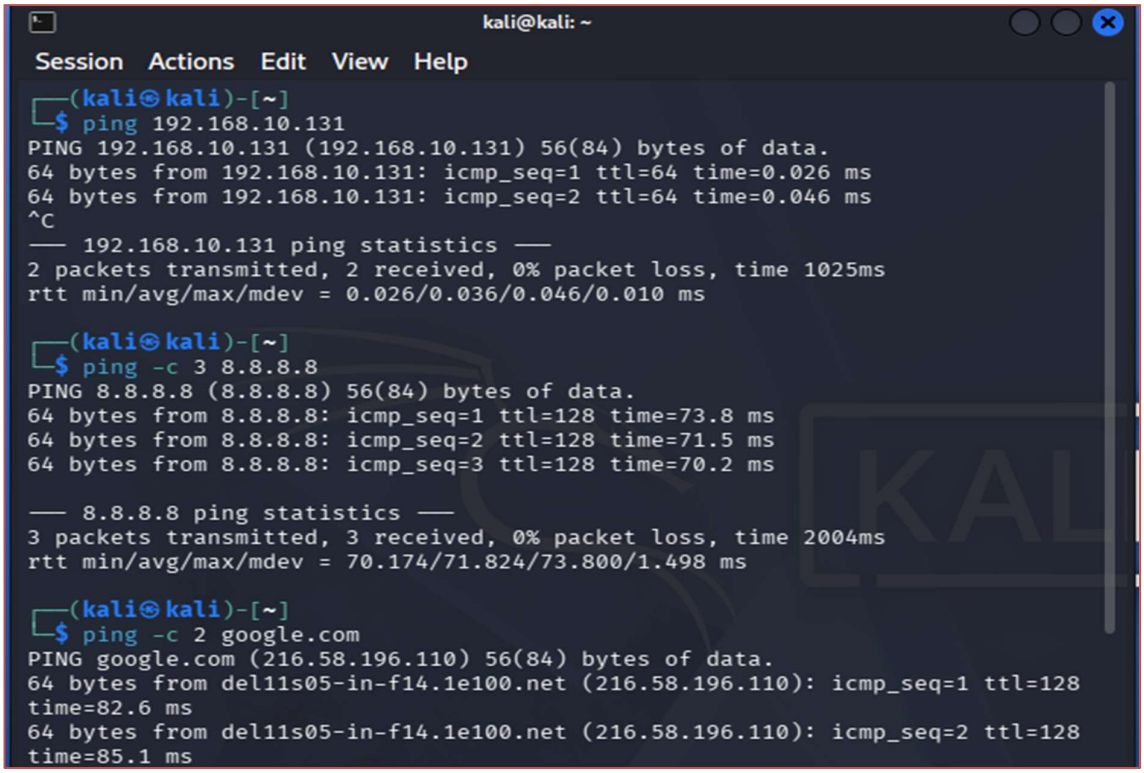


Figure 6 – Successful pings from Kali to 192.168.10.131, 8.8.8.8, and google.com after fixing the route and interface configuration.

6. SIEM Setup with ELK and Elastic Agent

6.1 ELK Stack on the Server

On the ELK-SERVER VM I installed Elasticsearch and Kibana and enabled them as system services. After installation, systemctl status elasticsearch and systemctl status kibana both showed active (running), confirming that the core SIEM components were up. Kibana was reachable from a browser using the ELK server IP and default port, which provided the central UI for managing agents and dashboards.

```
● elasticsearch.service - Elasticsearch
     Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-03-28 16:14:01 IST; 7min ago
       Docs: https://www.elastic.co
   Main PID: 1357 (java)
      Tasks: 143 (limit: 6946)
     Memory: 3.1G (peak: 4.2G swap: 1.5G swap peak: 1.8G)
        CPU: 4min 25.533s
     CGroup: /system.slice/elasticsearch.service
             ├─1357 /usr/share/elasticsearch/jdk/bin/java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server -Dcli.script=/usr/share/elasticsearch/bin/elasticsearch -Dcli.libs=lib/tools/server-cli -Des.path.home=/usr/share/elasticsearch -
             ├─1555 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -Des.networkaddress.cache.negative.ttl=10 -XX:+AlwaysPreTouch -Xss1m -Djava.awt.headless=true -Dfile.encoding=UTF-8 -Djna.nosys=true -XX:-OmitStr
             └─2390 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/controller

Mar 28 16:12:53 ELK-SERVER systemd[1]: Starting elasticsearch.service - Elasticsearch...
Mar 28 16:13:22 ELK-SERVER systemd-entrypoint[1357]: WARNING: Unknown module: jdk.internal.vm.ci specified to --add-exports
Mar 28 16:13:22 ELK-SERVER systemd-entrypoint[1357]: WARNING: Unknown module: org.elasticsearch.entitlement.instrumentationjava.logging specified to --add-exports
Mar 28 16:14:01 ELK-SERVER systemd[1]: Started elasticsearch.service - Elasticsearch.

● kibana.service - Kibana
     Loaded: loaded (/usr/lib/systemd/system/kibana.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-03-28 16:12:53 IST; 8min ago
       Docs: https://www.elastic.co
   Main PID: 1360 (node)
      Tasks: 11 (limit: 6946)
     Memory: 786.3M (peak: 1.4G swap: 43.6M swap peak: 65.8M)
        CPU: 1min 38.306s
     CGroup: /system.slice/kibana.service
             └─1360 /usr/share/kibana/bin/../node/glibc-217/bin/node /usr/share/kibana/bin/../src/cli/kibana/dist

Mar 28 16:17:38 ELK-SERVER kibana[1360]: [2026-03-28T16:17:38.635+05:30][INFO ][plugins.cloudSecurityPosture] Cloud Security telemetry: CSPM Cloud Connector Usage payload was sent successfully
Mar 28 16:17:38 ELK-SERVER kibana[1360]: [2026-03-28T16:17:38.682+05:30][INFO ][plugins.cloudSecurityPosture] Cloud Security telemetry: Muted Rules payload was sent successfully
Mar 28 16:17:38 ELK-SERVER kibana[1360]: [2026-03-28T16:17:38.694+05:30][INFO ][plugins.securitySolution] Fetch risk engine metrics
Mar 28 16:17:38 ELK-SERVER kibana[1360]: [2026-03-28T16:17:38.859+05:30][ERROR][plugins.securitySolution] Error fetching value lists overview metrics: Cannot read properties of undefined (reading 'by_type')
Mar 28 16:17:38 ELK-SERVER kibana[1360]: [2026-03-28T16:17:38.919+05:30][INFO ][plugins.cloudSecurityPosture] Cloud Security telemetry: Installation payload was sent successfully
Mar 28 16:17:38 ELK-SERVER kibana[1360]: [2026-03-28T16:17:38.923+05:30][ERROR][plugins.securitySolution] Error fetching value list items metrics: Cannot read properties of undefined (reading 'max_items_per_list')
Mar 28 16:17:39 ELK-SERVER kibana[1360]: [2026-03-28T16:17:39.618+05:30][INFO ][plugins.cloudSecurityPosture] Cloud Security telemetry: Indices payload was sent successfully
Mar 28 16:19:22 ELK-SERVER kibana[1360]: [2026-03-28T16:19:22.626+05:30][WARN ][plugins.cloudConnect] No API key found in saved object
Mar 28 16:20:25 ELK-SERVER kibana[1360]: [2026-03-28T16:20:25.880+05:30][INFO ][plugins.fleet.fleet:auto-install-content-packages-task:1.0.3] [AutoInstallContentPackagesTask] Fetched content packages discovery map: {"aws.cloudtrail.otel
Mar 28 16:20:25 ELK-SERVER kibana[1360]: [2026-03-28T16:20:25.907+05:30][INFO ][plugins.fleet.fleet:auto-install-content-packages-task:1.0.3] [AutoInstallContentPackagesTask] Found datasets with data:
 ESCOD
```

Figure 7 – Elasticsearch and Kibana services running successfully on the ELK-SERVER VM (service status output).

6.2 Fleet and Agent Configuration

Next, I set up Fleet in Kibana and enrolled two Elastic Agents:

- One agent on the ELK-SERVER itself, using a policy named **ELK-fleet-server-policy**, which runs the Fleet Server integration and keeps the other agents under central management.
- One agent on the Linux Mint victim (**mint-victim**) to forward system and authentication logs.

On mint-victim, the agent was attached to a policy called **mint-system-policy**, which includes the **System** integration with both “System logs” and “System auth” datasets enabled. After enrolment, the Agents page in Kibana showed the mint-victim agent as **Healthy**, and events from system.auth started appearing in the logs index.

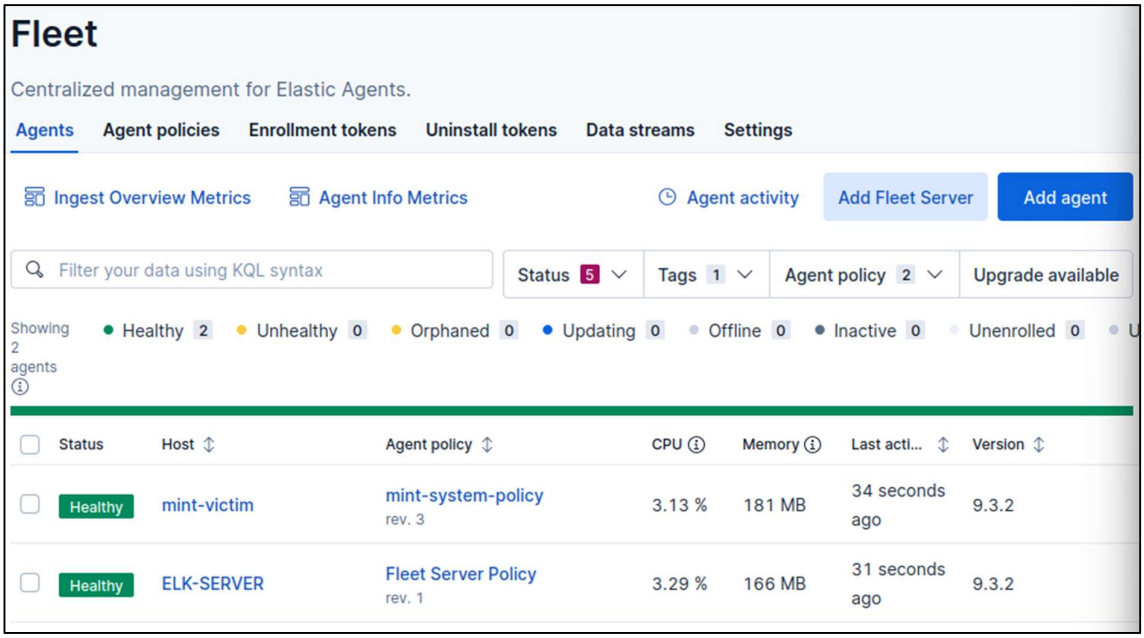


Figure 8 – Elastic Agent on mint-victim enrolled via Fleet and showing Healthy status under the mint-system-policy in Kibana.

7. SSH Brute-Force Incident and Network Forensics

7.1 Launching the Hydra Attack

To simulate a real incident, I used Hydra on the Kali VM to perform an SSH brute-force attack against mint-victim:

- Target: ssh://victim@192.168.10.132:22
- Source IP: 192.168.10.131 (Kali attacker)
- Tool: hydra -l victim -P <wordlist> 192.168.10.132 ssh

Hydra generated a series of rapid login attempts using the username **victim** and a list of common passwords. On the Kali terminal, Hydra clearly listed each attempt and marked them as failed, indicating that no valid password was found during the test.

```
┌──(kali㉿kali)-[~]
└─$ hydra -l victim -P /usr/share/wordlists/rockyou.txt ssh://192.168.10.132
-t 4
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
 military or secret service organizations, or for illegal purposes (this is n
on-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-29 02:
08:00
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip
 waiting)) from a previous session found, to prevent overwriting, ./hydra.res
tore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p
:14344399), ~3586100 tries per task
[DATA] attacking ssh://192.168.10.132:22/
[STATUS] 76.00 tries/min, 76 tries in 00:01h, 14344323 to do in 3145:42h, 4 a
ctive
[STATUS] 74.67 tries/min, 224 tries in 00:03h, 14344175 to do in 3201:50h, 4
active
^CThe session file ./hydra.restore was written. Type "hydra -R" to resume ses
sion.

┌──(kali㉿kali)-[~]
└─$
```

Figure 10 – Hydra brute-force attack from Kali against SSH on 192.168.10.132 using the victim username and a password wordlist.

7.2 Wireshark View of the Attack

While Hydra was running, I captured traffic on the victim's host-only interface with Wireshark using the filter tcp.port == 22.
The capture showed:

- TCP handshakes from 192.168.10.131 to 192.168.10.132 on port 22.
- Encrypted SSH packets representing repeated authentication attempts.

Each new attempt re-used the same attacker IP and destination port, which is typical brute-force behaviour against an SSH service.

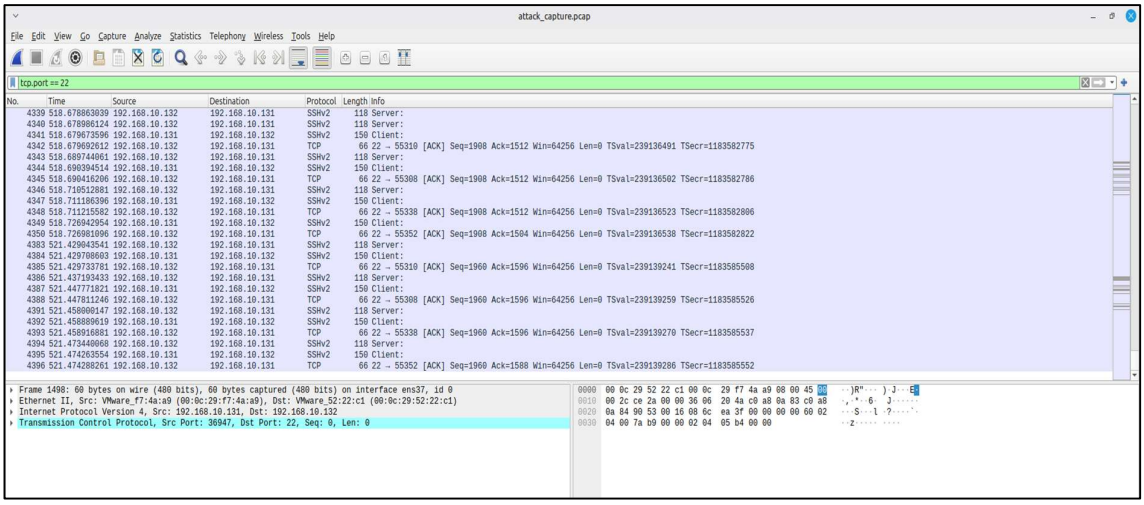


Figure 11 – Wireshark capture on mint-victim showing repeated SSH connections on tcp.port 22 from attacker IP 192.168.10.131.

7.3 Log Evidence in Kibana

In Kibana Discover, I filtered for SSH authentication failures with the following query:

process.name: "sshd" AND event.dataset: "system.auth" AND event.outcome: "failure"

During the Hydra time window, Discover showed a burst of sshd failure events where:

- source.ip = 192.168.10.131
- host.name = mint-victim
- user.name = victim

The messages clearly indicated “authentication failure” for the victim account, matching the attack pattern from Hydra. This proves that the SIEM captured the incident from the log side while Wireshark captured it from the packet side.

Figure 12 – Kibana Discover view highlighting a burst of sshd authentication failures from source.ip 192.168.10.131 targeting user victim on mint-victim.

8. Threat Hunting with Kibana

8.1 SSH Brute-Force Monitoring Dashboard

To make the attack easier to monitor, I built a small Kibana dashboard focusing on SSH activity:

- **Failed SSH logins over time (mint-victim):**
  A time-series chart counting events where process.name: "sshd" and event.outcome: "failure". The Hydra attack shows up as a sharp spike, making the brute-force attempt visually obvious.

- **Top attacker IPs (failed SSH):**
  A bar chart grouped by source.ip for the same filter. The top bar is 192.168.10.131 with the highest number of failures.

- **Top targeted usernames:**
  Another bar chart grouped by user.name, showing that “victim” is the main user under attack.

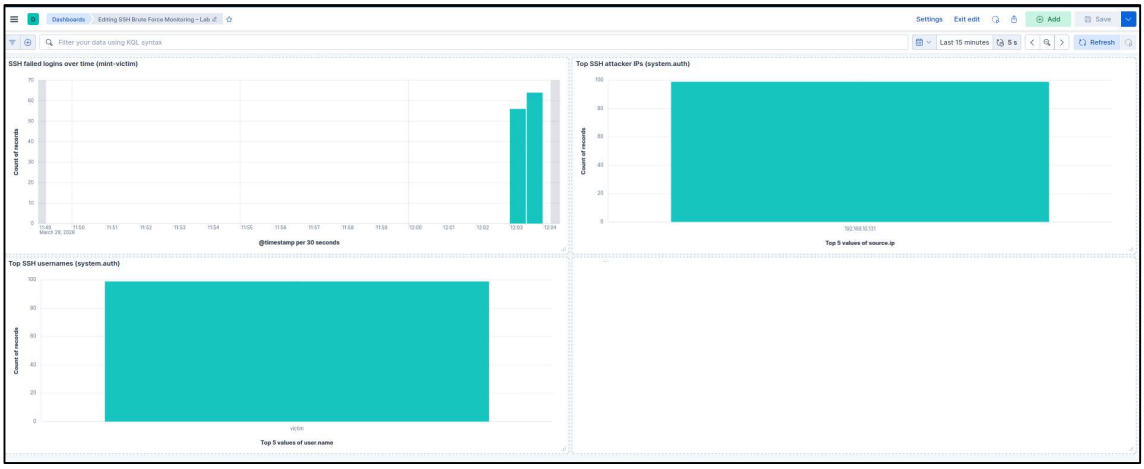


Figure 13 – Kibana dashboard panels showing a spike in failed SSH logins, top attacker IPs, and targeted usernames during the Hydra brute-force.

8.2 Checking for Successful Compromise

As a final step, I checked whether any of the brute-force attempts actually succeeded by running a second query:

process.name: "sshd" AND event.dataset: "system.auth" AND event.outcome: "success"

Within the Hydra time range, this query returned no results for source.ip: 192.168.10.131. This confirms that the attacker did not obtain a valid password, even though the log and packet data clearly show the brute-force behaviour. In a production SOC, this combination—visible brute-force with no success—would still justify blocking the attacker IP and possibly raising an alert for follow-up investigation.

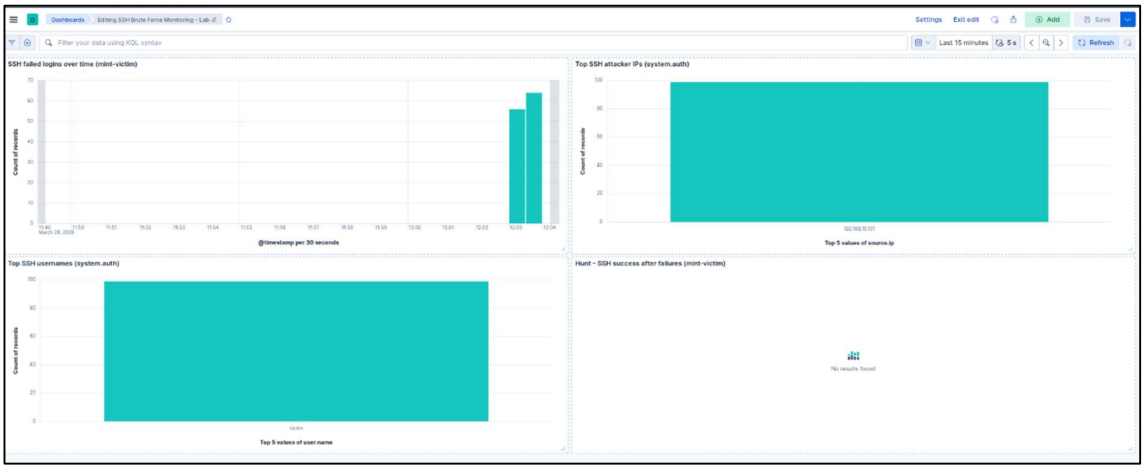


Figure 14 – Kibana panel showing no SSH success events from 192.168.10.131 during the Hydra attack time window.

9. Key Learnings and Challenges

During this task I connected network design, packet-level analysis, and SIEM monitoring into one workflow instead of treating them as separate topics. Designing the 192.168.1.0/27 office subnet forced me to think about host requirements, address planning, and basic connectivity tests, while the Packet Tracer lab and ICMP pings proved that the design actually works in a simulated office environment.

Working with Wireshark on the host-only network made it much clearer how normal traffic looks in terms of protocols and packet rates, and how short bursts or spikes stand out on the I/O Graph even when the overall traffic level is low. The troubleshooting mini-lab on Kali reinforced an important lesson: error messages like “Network is unreachable” are only the first clue; checking routes, interface status, and packet captures together is the fastest way to separate local misconfigurations from real network outages.

On the SIEM side, setting up Elasticsearch, Kibana, and Elastic Agent showed how important good field parsing is for detection and hunting. Once sshd events were coming in with clean fields such as source.ip, user.name, and event.outcome, it became easy to build dashboards that highlight brute-force behaviour and to write queries that answer specific questions like “Did the attacker ever succeed?”. Correlating the Hydra brute-force in both Wireshark and Kibana also made it very obvious how the same incident appears from a packet perspective versus a log perspective.

The main challenges were getting the Elastic Agents enrolled correctly through Fleet, keeping time windows aligned between Wireshark captures and Kibana queries, and controlling the Kali routing changes so that the “broken” and “fixed” states were both reproducible for screenshots. Each of these issues was solved by going back to basics—checking service status, confirming IP addresses and routes, and using simple filters in Discover to verify that the expected data was actually being ingested.